



Universidad Nacional Autónoma de México
ENES Juriquilla
Ingeniería Aeroespacial 2002
Guía práctica de estudio 02
Prof. Ing Andrés David Flores Ferro
2do semestre
Fecha de entrega: 14 de febrero del 2025

SOLUCIÓN DE PROBLEMAS Y ALGORITMOS

González Flores Daniela
No. de cuenta: 42504198-3
Luna Guerrero Axel Emilio
32218587-8
Quiroz Lara Erik
32225116-0

Fecha de revisión: 02 / 03 / 2025
Calificación: 98 de 100 pts
Observaciones:

- Podría mejorar un poco más la conexión de investigación con lo de clase, pero aceptable
- Agregar dato de tipo de archivo consultado en referencias [Web, PDF, Libro, etc] , no se baja por esta razón si no porque falta citar algo del contenido presentado en clase.
- En el desarrollo se describe todo lo que hizo pero no se atendieron correcciones de clase.

Actividad en clase 25 de 25 pts	
Protocolo de práctica	73 de 75 pts
Portada	10 de 10 pts
Introducción	15 de 15 pts
Desarrollo	23 de 25 pts
Conclusiones	15 de 15 pts
Referencias	10 de 10 pts
CALIFICACIÓN TOTAL Entrega 98 pts	

OBJETIVO

El alumno elaborará algoritmos correctos y eficientes en la solución de problemas siguiendo las etapas de Análisis y Diseño pertenecientes al Ciclo de vida del software.

PROPÓSITO ACTIVIDAD DESEADA

- A partir del enunciado de un problema, identificar el conjunto de entrada y el conjunto de salida.
- Elaborar un algoritmo que resuelva un problema determinado (dado por el profesor), identificando los módulos de entrada, de procesamiento y de salida.

INTRODUCCIÓN

La Real Academia Española, define problema como un “planteamiento de una situación cuya respuesta desconocida debe obtenerse a través de métodos científicos”.

Un problema informático es aquel que tiene que resolverse con un lenguaje de programación, utilizando técnicas de ingeniería de software que nos ayudan a crear el modelo de una situación real, permitiendo automatizar un proceso utilizando un programa de cómputo.

Para poder resolver un problema se debe establecer un enunciado coherente y claro que permita describir la situación que se busca resolver, identificando con qué datos se encuentra (entradas), que se desea obtener (salidas) y permite establecer qué interrelaciones existen entre la información y qué procesos son factibles de realizar con una computadora para obtener la solución (proceso)

El pensamiento algorítmico es una habilidad clave en informática que puede ser desarrollada independientemente de aprender programación.

Una correcta visualización de los problemas puede ayudar a entender conceptos básicos relacionados con algoritmos, eficiencia, conjuntos de datos, entre otras cosas.

Un algoritmo es un método de resolución de problemas que consiste en instrucciones definidas exactamente.

Los algoritmos son conformados por 3 módulos clave:

- Módulo de entrada: Se obtienen los datos que se requieren para resolver el problema, estos datos pueden ser ingresados por el usuario, provenientes de una base de datos o leer de un archivo.
- Módulo de procesamiento: Son las operaciones necesarias para obtener un resultado a la problemática dada.
- Módulo de salida: Permite mostrar los resultados a partir del módulo de procesamiento de datos, y los resultados se pueden mostrar en pantalla, archivos o una base de datos.

Ciclo de vida del software:

El ciclo de vida del software es un proceso que se sigue para desarrollar y mantener software de manera efectiva. Este ciclo se compone de varias fases que se llevan a cabo secuencialmente, con el fin de asegurarse de que se cumplan los requisitos y objetivos que se establecen al inicio del proyecto.

Fases del ciclo de vida del software:

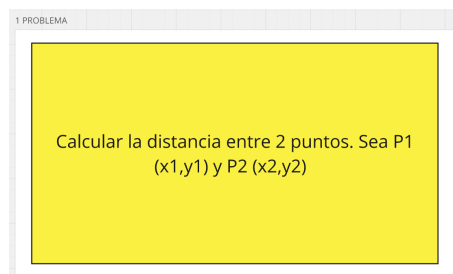
1. Planificación: Se define el alcance del proyecto, objetivos y requisitos del software.
2. Análisis: Se analizan detalladamente los requisitos y se identifican posibles problemas.
3. Diseño: Se diseña la arquitectura de software. Se crean los diagramas y modelos necesarios.
4. Implementación: Se desarrolla el software siguiendo los diseños previamente establecido y se realizan pruebas para comprobar su funcionamiento
5. Pruebas: Se realizan pruebas para comprobar que el software funciona correctamente
6. Despliegue: el software es instalado y configurado en donde se va a utilizar
7. Mantenimiento: Se llevan a cabo tareas de mantenimiento para asegurar el correcto funcionamiento del software y se realizan actualizaciones

DESARROLLO

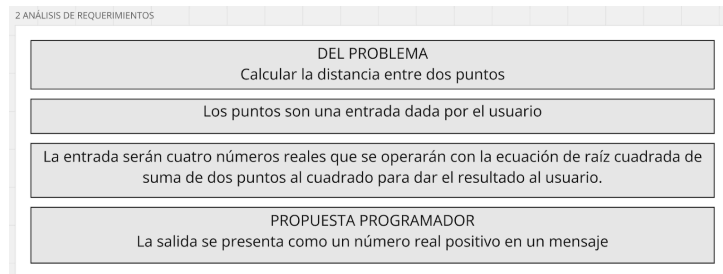
En orden para desarrollar y poner en práctica la elaboración de algoritmos como metodología para la resolución de problemas utilizamos dicha metodología para resolver dos problemas en concreto, siendo uno determinar la distancia entre dos puntos dados por un usuario y como segundo problema ordenar 3 números dados por un usuario de mayor a menor.

Problema 1: Calcular la distancia entre 2 puntos. Sea $P1(x1, y1)$, $P2(x2, y2)$

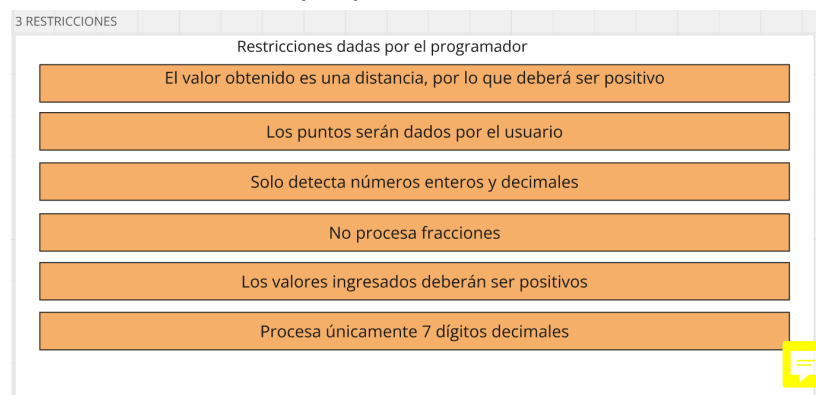
Como primer paso se tiene que definir cuál es el problema que se busca resolver.



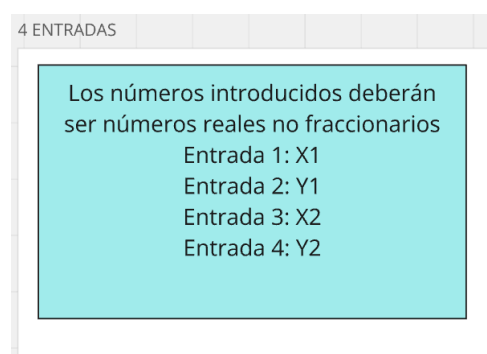
El segundo paso es definir que no está pidiendo el problema así como las necesidades que como programadores podemos identificar que se utilizarán a lo largo del proceso para poder resolver de modo eficaz el problema partiendo de las herramientas con las que contamos.



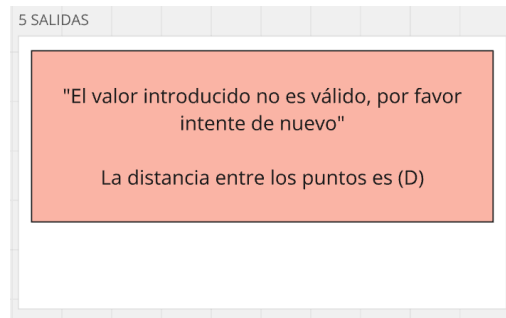
El tercer paso es definir las restricciones que existen en torno al problema y que servirán en un futuro como cotas que pueden afectar el funcionamiento .



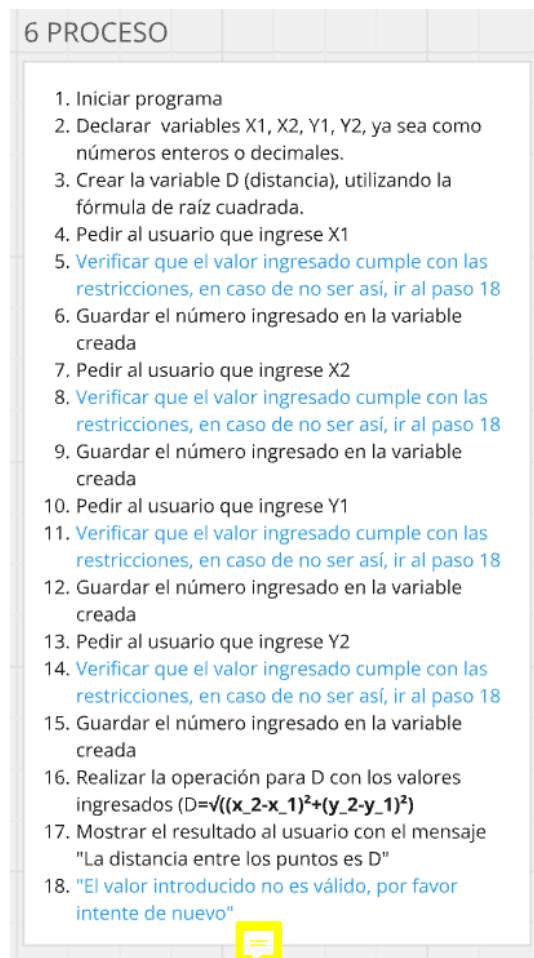
El cuarto paso consiste en definir las variables que se tomarán como entradas y en torno a las cuales el programa procesará para obtener un resultado que dé solución al problema.



En el quinto paso se definirán las salidas, las cuales son la información que se regresará al usuario como resultado del proceso de nuestro programa.



El sexto paso es desarrollar el proceso que realizará el programa paso a paso para resolver el problema en función de las entradas.



Por último y en orden para comprobar que nuestro proceso fue desarrollado correctamente se realizan las pruebas de escritorio para comprobar de forma analógica que de seguir paso a paso cada uno de los pasos del proceso con entradas hipotéticas se obtendrán salidas correctas, determinando así que el algoritmo es funcional para la resolución del problema en cuestión.

7 PRUEBAS DE ESCRITORIO		
ITERACIÓN	ENTRADA	SALIDA
1	X1=1 X2=3 Y1=4 Y2=5	2.24
2	X1=-9 X2=0 Y1=-8 Y2=-5	9.49
3	X1=1 X2=d Y1=7 Y2=6	"El valor introducido no es válido, por favor intente de nuevo"
4	X1=1/2 X2=9 Y1=0 Y2=1	"El valor introducido no es válido, por favor intente de nuevo"

Problema 2: Ordenar 3 números de mayor a menor, ingresados por el usuario.

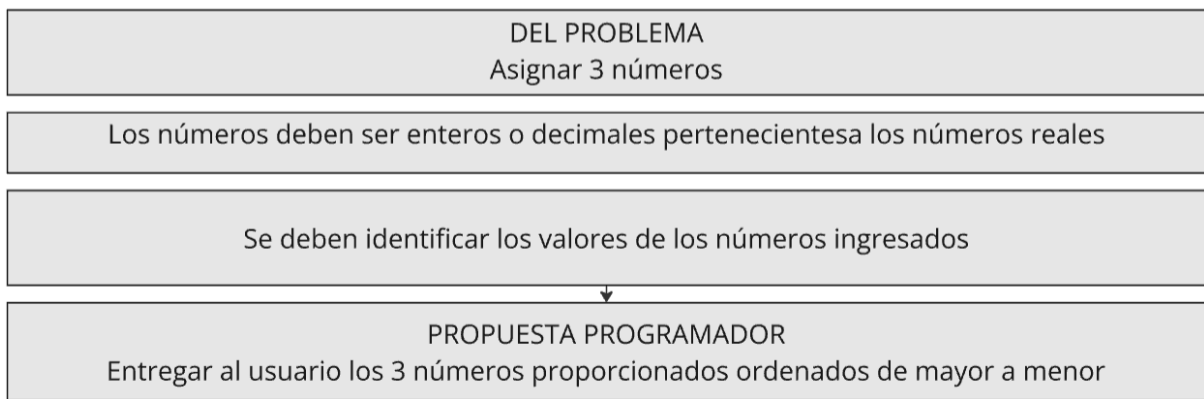
Al igual que en el problema 1, se presenta el mismo procedimiento para la resolución de la problemática.

Problema

Ordenar 3 números de mayor a menor,
ingresados por el usuario

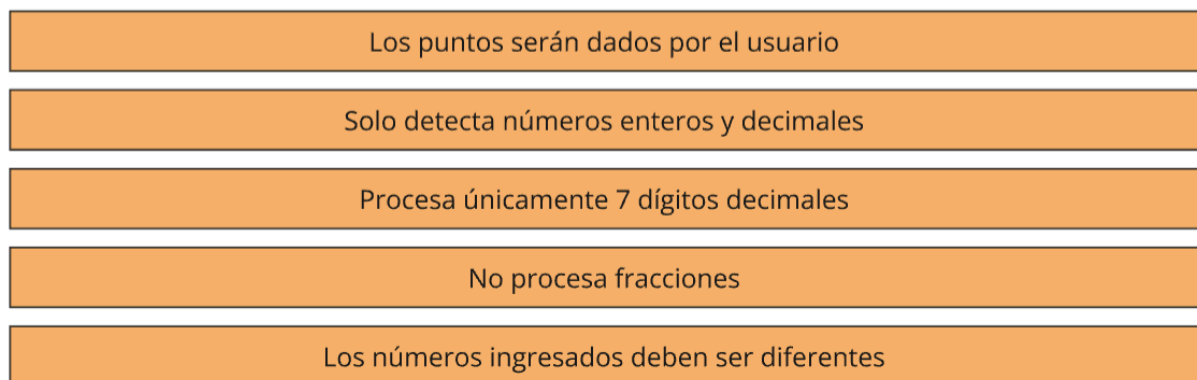
Análisis de requerimientos

Se realiza un análisis de requerimientos distinguiendo entre los que están establecidos en el problema y aquellos que son sugeridos por el programador.



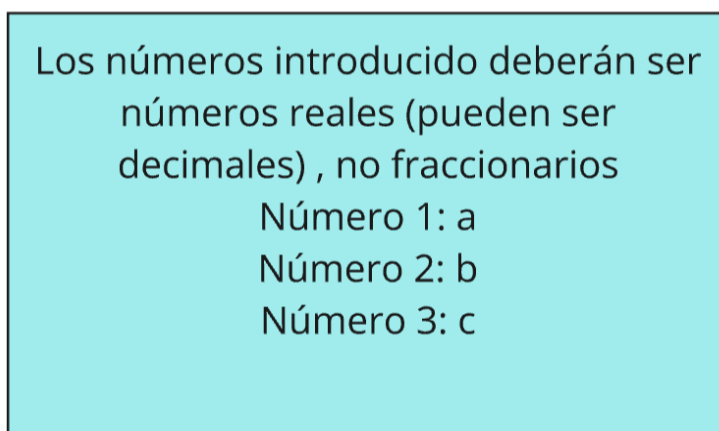
Restricciones

Restricciones dadas por el programador



Entradas

Se establece cuáles son los datos que tenemos o nos dará el usuario



Salidas

Qué es lo que le diremos al usuario, que le pediremos o sugeriremos.

"El valor introducido no es válido, por favor
intente de nuevo"

"El número mayor es : x"
"El segundo número mayor es: y"
"El número menor es: z"

Proceso

Se establece un paso a paso de lo que es necesario realizar para la resolución del problema.

1. Declarar variable a, ya sea como número entero o decimal.
2. Declarar variable b, ya sea como número entero o decimal.
3. Declarar variable c, ya sea como número entero o decimal.
4. Declarar variable x, ya sea como número entero o decimal.
5. Declarar variable y, ya sea como número entero o decimal.
6. Declarar variable z, ya sea como número entero o decimal.
7. Pedir al usuario que ingrese el número 1
8. Leer el número 1
9. Asignar el número 1 a la variable a
10. Verificar que la variable cumpla con los criterios
11. En caso de no cumplir con los criterios mostrar el texto "El valor introducido no es válido, por favor intente de nuevo" y regresar al paso 8
12. Guardar variable a
13. Pedir al usuario que ingrese el número 2
14. Leer el número 2
15. Asignar el número 2 a la variable b
16. Verificar que la variable b cumpla con los criterios
17. En caso de no cumplir con los criterios mostrar el texto "El valor introducido no es válido, por favor intente de nuevo" y regresar al paso 14
18. Guardar variable b
19. Pedir al usuario que ingrese el número 3
20. Leer el número 3
21. Asignar el número 3 a la variable c
22. Verificar que la variable c cumpla con los criterios
23. En caso de no cumplir con los criterios mostrar el texto "El valor introducido no es válido, por favor intente de nuevo" y regresar al paso 20
24. Guardar variable c
25. Comparar a con b
26. Determinar al mayor con x y menor con y
27. Comparar b con c
28. Determinar si b es mayor que c, asignar c a z
29. En caso de c ser mayor que b comparar a con c
30. Asignar b a z
31. Asignar al mayor entre a y b; x y al menor y
32. Mostrar el resultado al usuario
33. "El número mayor es: x"
34. "El segundo número mayor es: y"
35. "El número menor es: z"
36. Finalizar programa.

Pruebas de escritorio

Cuales son las posibles entradas y salidas y en qué casos nuestro código podría tener fallas.

ITERACIÓN	ENTRADA	SALIDA
1	2, 7, 5	2, 5, 7
2	3,-6,0	-6, 0, 3
3	1,2,m	"El valor introducido no es válido, por favor intente de nuevo"
4	3, 4, 1/4	"El valor introducido no es válido, por favor intente de nuevo"
5	3, 3, 3	"El valor introducido no es válido, por favor intente de nuevo"

Conclusión:

Axel:

El desarrollo de algoritmos es una herramienta fundamental para la resolución de problemas en informática, ya que permite estructurar soluciones eficientes mediante un conjunto de pasos bien definidos. A través de la aplicación del ciclo de vida del software, es posible garantizar que el análisis, diseño y desarrollo de un algoritmo se realicen de manera ordenada y eficaz.

En los problemas abordados, se siguió un proceso sistemático que incluyó la identificación de entradas y salidas, el análisis de restricciones, la definición de variables y el desarrollo del proceso paso a paso. Finalmente, mediante pruebas de escritorio, se verificó el correcto funcionamiento de los algoritmos.

Con base en lo anterior, se reafirma la importancia del pensamiento algorítmico como una habilidad esencial en el ámbito de la computación, permitiendo modelar situaciones del mundo real y optimizar su solución a través de herramientas computacionales.

Daniela:

El pensamiento algorítmico es una habilidad clave para la resolución de problemas de programación, ya que nos permite identificar adecuadamente el problema y que es lo que la solución debe contener para ser correcta y eficiente.

Resolver problemas de esta manera nos permite ordenar la información que tenemos, que es lo que queremos y que podemos obtener, así como resolver oportunamente los problemas que se pudieran presentar al crear un código.

En esta práctica se lograron plantear dos algoritmos correctos y eficientes para la solución de los problemas planteados en clase siguiendo las correspondientes etapas para el ciclo de vida del software, por lo que se cumplieron los objetivos establecidos. La práctica fue útil para la comprensión de los conceptos vistos en clase.

Pudimos aplicar los conceptos vistos en clase en la realización de la práctica y complementarlos al ir desarrollando la práctica y realizando investigación posteriormente.

Erik:

Considero que la forma de abordar esta metodología es de gran utilidad tanto para la rama de programación como lo es para la solución de problemas en otros entornos de forma precisa y detallada.

De igual manera considero que la práctica y sus ejercicios con problemas a resolver fueron los necesarios para trabajar la lógica que hay detrás de cada acción paso a paso por más simple que sea debido a la utilidad que esto puede llegar a tener en la eficacia de trabajo en proyectos de mayor volumen.

La teoría así como el acompañamiento durante la práctica fueron los suficientes y necesarios para conseguir resolver y completar los objetivos de esta práctica.

REFERENCIAS

Ferro, A. D. F. (febrero del 2025). *Resolución de problemas*.

Futschek, G.

(2006). Algorithmic Thinking: The Key for Understanding Computer Science. *Lecture Notes in Computer Science*, 4226.

(Gálvez, s/f)

Gálvez, J. A. S. (s/f). *Solución de Problemas y Algoritmos*. Unidades de Apoyo para el Aprendizaje - CUAIEED - UNAM. Recuperado el 13 de febrero de 2025, de <https://uapa.cuaed.unam.mx/sites/default/files/minisite/static/3fa4df86-0459-4c54-a569-87d7c387f449/UAPA-solucion-problemas-algoritmos/index.html>

Sánchez, C. E. R., & Ramos, A. Á. (2022, mayo 1). *Problemas y sus elementos*. Portal Académico del CCH. <https://portalacademico.cch.unam.mx/cibernetica1/metodologia-resolucion-problemas/problemas-elementos>

Martínez, S. (2023, abril 25). Ciclo de vida del software: qué es, etapas y modelos. *Inesdi*. <https://www.inesdi.com/blog/ciclo-de-vida-del-software/>

